

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

KESHAV TOSHNIWAL (1BF24CS135)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)**

BENGALURU-560019

February to June 2026

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by KESHAV TOSHNIWAL (**1BF24CS135**), who is a bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Dr. Manjula S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write a program to obtain the Topological ordering of vertices in a given digraph.	4-7
2	Implement Johnson Trotter algorithm to generate permutations.	8-9
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	10-13
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	14-17
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	18-20
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	21-22
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	23-26
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	27-28 28-30
9	Implement Fractional Knapsack using Greedy technique.	31-34
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	35-36
11	Implement "N-Queens Problem" using Backtracking.	37-38

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Topological Sorting

```
#include <stdio.h>

#define MAX 10

int graph[MAX][MAX], indegree[MAX], queue[MAX];
int n;

void topologicalSort() {
    int front = 0, rear = 0;
    int count = 0, i, j;
    // Find indegree of each vertex
    for (i = 0; i < n; i++) {
        indegree[i] = 0;
        for (j = 0; j < n; j++)
            indegree[i] += graph[j][i];
    }
    // Insert all vertices with indegree 0 into queue
    for (i = 0; i < n; i++)
        if (indegree[i] == 0)
            queue[rear++] = i;
    printf("Topological Ordering: ");
    while (front < rear) {
        int v = queue[front++];
        printf("%d ", v);
        count++;
        // Reduce indegree of adjacent vertices
        for (i = 0; i < n; i++)
            if (graph[v][i] == 1) {
                indegree[i]--;
                if (indegree[i] == 0)
                    queue[rear++] = i;
            }
    }
}
```

```

// Check for cycle
if (count != n)
    printf("\nGraph contains a cycle. Topological sort not possible.\n");
}

int main() {
    int i, j;

    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);
    topologicalSort();

    return 0;
}

```

OUTPUT:

```

Enter number of vertices: 6
Enter adjacency matrix:
0 1 1 0 0 0
0 0 0 1 1 0
0 0 0 1 0 0
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0
Topological Ordering: 0 1 2 4 3 5

```

LEETCODE 207

207. Course Schedule Solved

Medium Topics Companies Hint

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you **must** take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`
Output: `true`
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: `numCourses = 2, prerequisites = [[1,0],[0,1]]`
Output: `false`
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- `1 <= numCourses <= 1000`
- `0 <= prerequisites.length <= 1000`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- `ai != bi`
- All the prerequisites pairs are unique.

Accepted 54 / 54 testcases passed
daku_keshav submitted at May 23, 2026 07:50

Runtime: 37 ms | Beats 34.78%
Memory: 109.89 MB | Beats 42.65%

Code | C

```
1 bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {  
2     // Allocate graph  
3     int** graph = (int**)malloc(numCourses * sizeof(int*));  
4     int* indegree = (int*)calloc(numCourses, sizeof(int));
```

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
```

```
    int** graph = (int**)malloc(numCourses * sizeof(int*));
```

```
    int* indegree = (int*)calloc(numCourses, sizeof(int));
```

```
    int* graphSize = (int*)calloc(numCourses, sizeof(int));
```

```
    for (int i = 0; i < numCourses; i++)
```

```
        graph[i] = (int*)malloc(numCourses * sizeof(int));
```

```
    for (int i = 0; i < prerequisitesSize; i++) {
```

```
        int course = prerequisites[i][0];
```

```
        int prereq = prerequisites[i][1];
```

```
        graph[prereq][graphSize[prereq]++] = course;
```

```
        indegree[course]++;
```

```
    }
```

```
    int* queue = (int*)malloc(numCourses * sizeof(int));
```

```
    int front = 0, rear = 0;
```

```
    for (int i = 0; i < numCourses; i++)
```

```
        if (indegree[i] == 0)
```

```
            queue[rear++] = i;
```

```
int count = 0;
while (front < rear) {
    int node = queue[front++];
    count++;

    for (int i = 0; i < graphSize[node]; i++) {
        int neighbor = graph[node][i];
        indegree[neighbor]--;
        if (indegree[neighbor] == 0)
            queue[rear++] = neighbor;
    }
}
return count == numCourses;
}
```

Lab program 2:

Johnson Trotter

```
#include <stdio.h>

// -1 means left

int mobile(int p[], int d[], int n){
    int m = 0;
    for (int i = 0; i < n; i++) {
        if (d[i] == -1 && i > 0 && p[i] > p[i - 1] && p[i] > m)
            m = p[i];
        if (d[i] == 1 && i < n - 1 && p[i] > p[i + 1] && p[i] > m)
            m = p[i];
    }
    return m;
}

int main(){
    int n;
    int p[10], d[10];
    int i, m, pos, next, temp;
    printf("Enter n:");
    scanf("%d", &n);
    for (i = 0; i < n; i++){
        p[i] = i + 1;
        d[i] = -1;
    }

    while (1)
    {
        for (i = 0; i < n; i++){
            printf("%d ", p[i]);
        }

        printf("\n");
        m = mobile(p, d, n);
```



```

    if (m == 0)
        break;
    for (i = 0; i < n; i++)
        if (p[i] == m)
            pos = i;
    next = pos + d[pos];
    temp = p[pos];
    p[pos] = p[next];
    p[next] = temp;

    temp = d[pos];
    d[pos] = d[next];
    d[next] = temp;

    for (i = 0; i < n; i++)
        if (p[i] > m)
            d[i] = -d[i];
}
return 0;
}

```

OUTPUT

```

Enter n:3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

```

Lab Program 3:

Merge Sort

```
#include<stdlib.h>
```

```
#include <stdio.h>
```

```
#include <time.h>
```

```
void merge (int low,int mid,int high, int *a){
```

```
    int i=low, j=mid+1,k=low;
```

```
    int arr[high+1];
```

```
    while(i<=mid && j<= high){
```

```
        if(a[i]< a[j]) arr[k++]=a[i++];
```

```
        else arr[k++]=a[j++];
```

```
    }
```

```
    while(i<=mid) arr[k++]=a[i++];
```

```
    while(j<=high) arr[k++]=a[j++];
```

```
    for(i=low;i<=high;i++) a[i]=arr[i];
```

```
}
```

```
void mergeSort(int low,int high, int *a){
```

```
    if(low == high) return;
```

```
    int mid =(low + high)/2;
```

```
    mergeSort(low,mid,a);
```

```
    mergeSort(mid+1,high,a);
```

```
    merge(low, mid, high, a);
```

```
}
```

```
int main(){
```

```
    int n;
```

```
    printf("Enter size :");
```

```
    scanf("%d",&n);
```

```
    int a[n];
```

```

// printf("Enter %d items:\n",n);
for(int i=0; i<n;i++)
    a[i]=rand();
    // scanf("%d",&a[i]);

double stTime =clock();
mergeSort(0,n-1,a);
double endTime =clock();
double t= (endTime -stTime)/CLOCKS_PER_SEC;
// printf("Sorted Array :\n");
// for(int i=0;i<n;i++)
//     printf("%d\t",a[i]);
printf("\nTime Taken: %.4f ms",t*1000);
return 0;
}

```

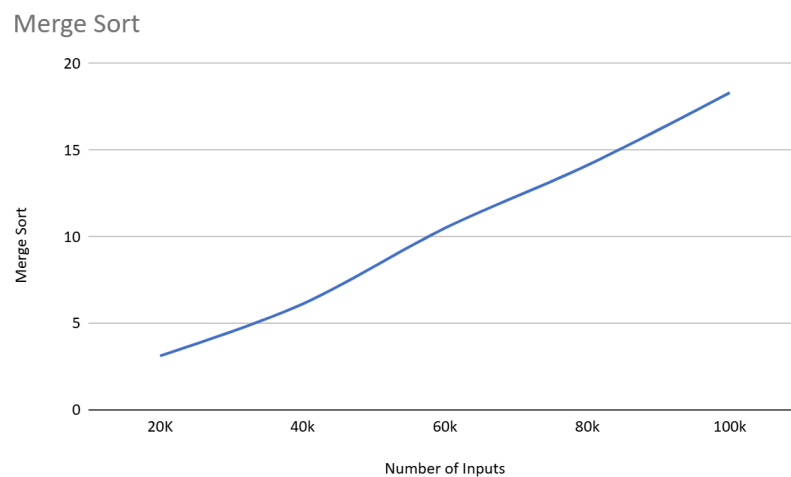
OUTPUT:

```

Enter size :5
Enter 5 items:
20 25 15 10 17
Sorted Array :
10 15 17 20 25
Time Taken: 0.0020 ms

```

GRAPH:



LEETCODE 148

148. Sort List

Medium Topics Companies

Given the *head* of a linked list, return the list after sorting it in *ascending order*.

Example 1:

Input: head = [4,2,1,3]
Output: [1,2,3,4]

Example 2:

Input: head = [-1,5,3,4,0]
Output: [-1,3,4,5,0]

Accepted 30 / 30 testcases passed
daku_keshav submitted at May 05, 2026 22:36

Runtime: 17 ms | Beats 26.83%
Memory: 25.58 MB | Beats 45.57%

Code | C

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     struct ListNode *next;
6  * };
7  */
8 struct ListNode* merge(struct ListNode * pehla , struct ListNode * dusra){
9     struct ListNode head;
10    struct ListNode * temp = &head ;
11    head.next = NULL ;
12
13    while(pehla && dusra){
14        if(pehla->val < dusra->val){
15            temp->next = pehla;
16            pehla = pehla->next;
17        }
18        else{
19            temp->next = dusra;
20            dusra = dusra->next;
21        }
22        temp = temp->next;
23    }
24    if(pehla){
25        temp->next = pehla;
26    }
27    if(dusra){
28        temp->next = dusra;
29    }
30    return head;
31 }
```

Testcase | Test Result

You must run your code first

/**

* Definition for singly-linked list.

* struct ListNode {

* int val;

* struct ListNode *next;

* };

*/

struct ListNode* merge(struct ListNode * pehla , struct ListNode * dusra){

struct ListNode head;

struct ListNode * temp = &head ;

head.next = NULL ;

while(pehla && dusra){

if(pehla->val < dusra->val){

temp->next = pehla;

pehla = pehla->next;

}

else{

temp->next = dusra;

dusra = dusra->next;

```

    }
    temp = temp->next;
}

temp->next= pehla ? pehla : dusra ;

return head.next;
}

struct ListNode* sortList(struct ListNode* head) {
    if(!head || !head->next) return head;

    struct ListNode * slow =head , *fast = head , * prev = NULL;
    while(fast && fast->next){
        prev = slow;
        slow = slow->next;
        fast = fast->next->next ;
    }

    prev-> next=NULL;
    struct ListNode* h = sortList(head);
    struct ListNode* s = sortList(slow);
    return merge(h,s);
}

```

Lab program 4:

Quick Sort

```
#include<stdlib.h>
```

```
#include <stdio.h>
```

```
#include <time.h>
```

```
void swap(int a[], int i,int j){
```

```
    int t=a[i];
```

```
    a[i]=a[j];
```

```
    a[j]=t;
```

```
}
```

```
int sort(int a[],int st,int end){
```

```
    int p=a[st],i=st,j=end;
```

```
    while (1) {
```

```
        while (i <= end && a[i] <= p) i++;
```

```
        while (j >= st && a[j] > p) j--;
```

```
        if (i >= j) break;
```

```
        swap(a, i, j);
```

```
    }
```

```
    swap(a,st,j);
```

```
    return j;
```

```
}
```

```
void quickSort(int a[],int st,int end ){
```

```
    if(st >= end) return;
```

```
    int mid = sort(a,st,end);
```

```
    quickSort(a,st,mid-1);
```

```
    quickSort(a,mid+1,end);
```

```
}
```

```
int main(){
```

```

int n;
printf("Enter size :");
scanf("%d",&n);
int a[n];
// printf("Enter %d items :\n", n);
for(int i=0; i<n;i++)
    a[i]=rand();
    // scanf("%d",&a[i]);

double stTime =clock();
quickSort(a,0,n-1);
double endTime =clock();
double t= (endTime -stTime)/CLOCKS_PER_SEC;
// printf("Sorted Array :\n");
// for(int i=0;i<n;i++)
    // printf("%d\t",a[i]);
printf("\nTime Taken: %.4f ms",t*1000);
return 0;
}

```

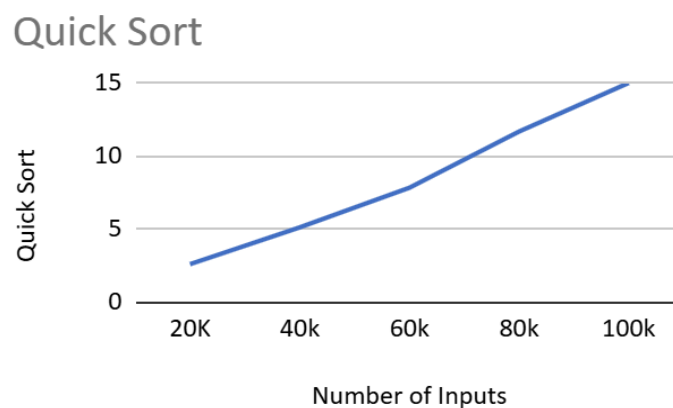
OUTPUT:

```

Enter size :5
Enter 5 items :
80 16 17 19 18
Sorted Array :
16 17 18 19 80
Time Taken: 0.0030 ms

```

GRAPH:



LEETCODE 455

455. Assign Cookies Solved

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor $g[i]$, which is the minimum size of a cookie that the child will be content with; and each cookie j has a size $s[j]$. If $s[j] \geq g[i]$, we can assign the cookie j to the child i , and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:
Input: $g = [1,2,3]$, $s = [1,1]$
Output: 1
Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3. And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content. You need to output 1.

Example 2:
Input: $g = [1,2]$, $s = [1,2,3]$
Output: 2
Explanation: You have 2 children and 3 cookies. The greed factors of 2 children are 1, 2. You have 3 cookies and their sizes are big enough to gratify all of the children. You need to output 2.

Runtime: 29 ms | Beats 13.44%
Memory: 12.52 MB | Beats 73.79%

Code | C

```
1 int compare(const void *a, const void *b) {
2     return (*(int*)a - *(int*)b);
3 }
4 int findContentChildren(int* g, int gSize, int* s, int sSize) {
```

Testcase | Test Result

You must run your code first

```
void swap(int a[], int i,int j){
    int t=a[i];
    a[i]=a[j];
    a[j]=t;
}
```

```
int sort(int a[],int st,int end){
    int p=a[st],i=st,j=end;
    while (1) {
        while (i <= end && a[i] <= p) i++;
        while (j >= st && a[j] > p) j--;

        if (i >= j) break;
        swap(a, i, j);
    }
    swap(a,st,j);
    return j;
}
```

```
void quickSort(int a[],int st,int end ){
```



```

    if(st >= end) return;

    int mid = sort(a,st,end);
    quickSort(a,st,mid-1);
    quickSort(a,mid+1,end);
}

int findContentChildren(int* g, int gSize, int* s, int sSize) {
    quickSort(g, 0, gSize-1);
    quickSort(s, 0, sSize-1);
    int count=0, i=0, j=0;
    while(i<gSize && j<sSize){
        if(g[i]<=s[j]) {
            count++;i++;j++;
        }
        else j++;
    }
    return count;
}

```

Lab program 5:

Heap Sort

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
void heapify(int arr[], int n, int i) {  
    int lar = i;  
    int left = 2 * i + 1;  
    int right = 2 * i + 2;  
  
    if (left < n && arr[left] > arr[lar])  
        lar = left;  
    if (right < n && arr[right] > arr[lar])  
        lar = right;  
    if (lar != i) {  
        int t = arr[i];  
        arr[i] = arr[lar];  
        arr[lar] = t;  
        heapify(arr, n, lar);  
    }  
}
```

```
void heapSort(int arr[], int n) {  
    for (int i = n / 2 - 1; i >= 0; i--)  
        heapify(arr, n, i);  
    for (int i = n - 1; i > 0; i--) {  
        int t = arr[0];  
        arr[0] = arr[i];  
        arr[i] = t;  
        heapify(arr, i, 0);  
    }  
}
```

```

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    if (n <= 0) {
        printf("Invalid number of elements.\n");
        return 1;
    }
    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }
    // printf("Enter %d integer elements:\n", n);
    for (int i = 0; i < n; i++)
        arr[i]=rand();
        // scanf("%d", &arr[i]);
    clock_t start = clock();
    heapSort(arr, n);
    clock_t end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    // printf("\nSorted elements:\n");
    // for (int i = 0; i < n; i++)
    //     printf("%d ", arr[i]);
    printf("\n\nTime taken: %.3f ms\n", time_taken*1000);
    return 0;
}

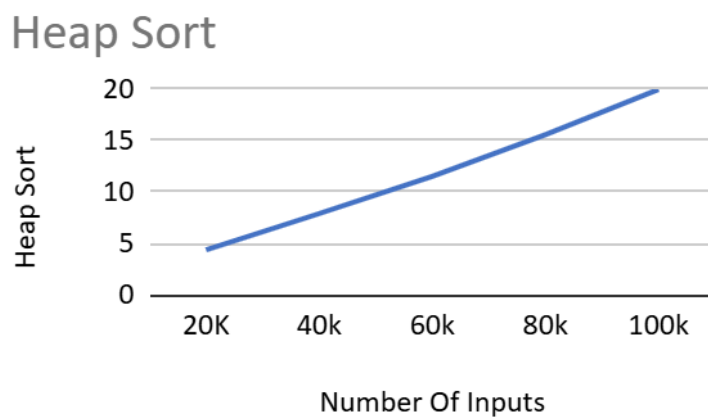
```

OUTPUT:

```
Enter number of elements: 7
Enter 7 integer elements:
15 10 20 17 8 25 5
|
Sorted elements:
5 8 10 15 17 20 25

Time taken: 0.002 ms
```

GRAPH:



Lab program 6:

Knapsack 0/1 (using DP)

```
#include <stdio.h>

int max(int a , int b){
    return (a>b)?a:b;
}

int knapsack(int W, int wt[], int val[], int n){
    int dp[n+1][W+1];
    for(int i=0;i<=n;i++)
        for(int w=0;w<=W;w++)
            if(i==0 || w==0) dp[i][w]=0;
            else if(wt[i-1]<=w) dp[i][w]=max(val[i-1]+dp[i-1][w-wt[i-1]], dp[i-1][w]);
            else dp[i][w]= dp[i-1][w];
    for(int i=0;i<=n;i++){
        for(int w=0;w<=W;w++)
            printf("%d\t",dp[i][w]);
        printf("\n");
    }
    return dp[n][W];
}

void main(){
    int n, W;
    printf("Enter number of items:");
    scanf("%d",&n);
    printf("Enter max capacity:");
    scanf("%d",&W);
    int val[n];
    int wt[n];
    printf("Enter wt and value for %d items:\n",n);
    for(int i=0;i<n;i++)
        scanf("%d %d",&wt[i], &val[i]);
    printf("Result = %d", knapsack(W, wt, val ,n));
}
```

OUTPUT:

```
Enter number of items:4
Enter max capacity:5
Enter wt and value for 4 items:
2 8
1 6
3 16
2 11
0 0 0 0 0 0
0 0 8 8 8 8
0 6 8 14 14 14
0 6 8 16 22 24
0 6 11 17 22 27
Result = 27
```

LEETCODE 70

70. Climbing Stairs Solved

Easy Topics Companies Hint

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

Input: $n = 2$
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps

Example 2:

Input: $n = 3$
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step

Constraints:

- $1 \leq n \leq 45$

24.6K 668 180 Online

All Submissions

Accepted 45 / 45 testcases passed
daku_keshav submitted at May 05, 2026 22:28

Analysis Solution

Runtime 0 ms | Beats 100.00%
Memory 8.56 MB | Beats 37.12%

Code | C

```
1 int climbStairs(int n) {
2     if (n <= 2) return n;
3     int dp[n + 1];
4     dp[1] = 1; dp[2] = 2;
```

Testcase Test Result

You must run your code first

```
int climbStairs(int n) {
    if (n <= 2) return n;
    int dp[n + 1];
    dp[1] = 1; dp[2] = 2;
    for (int i = 3; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}
```

Lab program 7:

Floyd's Algorithm

```
#include <stdio.h>

#define N 10

#define INFI 99999

int min(int a, int b){
    return (a<b)? a: b;
}

void floyd(int d[N][N], int n){
    for (int k=0;k<n;k++)
        for(int i=0;i<n;i++)
            for(int j=0;j<n;j++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

    printf("\nUpdated Path :\n");
    for (int i = 0; i < n; i++){
        for(int j = 0; j < n; j++)
            if (d[i][j] == INFI) printf("INF\t");
            else printf("%d\t", d[i][j]);
        printf("\n");
    }
}

void main(){
    int path[N][N];
    int n;
    printf("Enter number of nodes:");
    scanf("%d",&n);
    printf("Enter distance matrix\n(enter -1 for infinity):\n");
    for (int i=0;i<n;i++)
        for(int j=0;j<n;j++){
            int a;
            scanf("%d",&a);
            if(a==-1) path[i][j] = INFI;
```

```

        else path[i][j]= a;
    }
    floyd(path, n);
}

```

OUTPUT :

```

Enter number of nodes:4
Enter distance matrix
(enter -1 for infinity):
0 -1 3 -1
2 0 -1 -1
-1 7 0 -1
6 -1 -1 0

Updated Path :
0  10  3  INF
2  0  5  INF
9  7  0  INF
6  16  9  0

```

LEETCODE 3243

The screenshot displays the LeetCode interface for problem 3243. The problem description states: "You are given an integer n and a 2D integer array `queries`. There are n cities numbered from 0 to $n - 1$. Initially, there is a **unidirectional** road from city i to city $i + 1$ for all $0 \leq i < n - 1$. `queries[i] = [ui, vi]` represents the addition of a new **unidirectional** road from city u_i to city v_i . After each query, you need to find the **length** of the **shortest path** from city 0 to city $n - 1$. Return an array `answer` where for each i in the range $[0, \text{queries.length} - 1]$, `answer[i]` is the **length of the shortest path** from city 0 to city $n - 1$ after processing the **first** $i + 1$ queries." An example is provided: Input: $n = 5$, `queries = [[2,4],[0,2],[0,4]]`; Output: `[3,2,1]`. The submission status is 'Accepted' with 972/972 testcases passed. The runtime is 88 ms (Beats 40.00%) and memory usage is 21.56 MB (Beats 85.00%). A bar chart shows the runtime distribution across testcases.

```
/**
```

```
 * Note: The returned array must be malloced, assume caller calls free().
```

```
*/
```

```
#define MAXN 505
```

```
int adj[MAXN][MAXN];
```



```

int adjSize[MAXN];

int bfs(int n) {
    int dist[MAXN];
    for (int i = 0; i < n; i++) dist[i] = -1;

    int queue[MAXN];
    int front = 0, back = 0;

    queue[back++] = 0;
    dist[0] = 0;

    while (front < back) {
        int u = queue[front++];
        for (int i = 0; i < adjSize[u]; i++) {
            int v = adj[u][i];
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                queue[back++] = v;
            }
        }
    }

    return dist[n - 1];
}

int* shortestDistanceAfterQueries(int n, int** queries, int queriesSize, int* queriesColSize,
int* returnSize) {
    for (int i = 0; i < n; i++) adjSize[i] = 0;

    for (int i = 0; i < n - 1; i++)
        adj[i][adjSize[i]++] = i + 1;

    int* result = (int*)malloc(sizeof(int) * queriesSize);

```

```
for (int i = 0; i < queriesSize; i++) {  
    int u = queries[i][0];  
    int v = queries[i][1];  
  
    adj[u][adjSize[u]++] = v;  
    result[i] = bfs(n);  
}  
  
*returnSize = queriesSize;  
return result;  
}
```

Lab program 8a :

Prim's Algorithm

```
#include <stdio.h>
#define MAX 100
#define INF 999999

void prims(int cost[MAX][MAX], int n){
    int visited[MAX] = {0}, minCost = 0, edges = 0;
    visited[0] = 1;

    while (edges < n - 1){
        int min = INF, u, v;
        for (int i = 0; i < n; i++)
            if (visited[i])
                for (int j = 0; j < n; j++)
                    if (!visited[j] && cost[i][j] < min){
                        min = cost[i][j]; u = i; v = j;
                    }

        printf("%d - %d : %d\n", u, v, min);
        minCost += min;
        visited[v] = 1;
        edges++;
    }
    printf("Minimum Cost = %d\n", minCost);
}

void main(){
    int n, cost[MAX][MAX];
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the adjacency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++){
```

```

        scanf("%d", &cost[i][j]);
        if (i != j && cost[i][j] == 0) cost[i][j] = INF;
    }

    prims(cost, n);
}

```

OUTPUT:

```

Enter the number of vertices: 4
Enter the adjacency matrix:
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0
0 - 3 : 5
3 - 2 : 4
0 - 1 : 10
Minimum Cost = 19

```

Lab program 8b:

Kruskal's Algorithm

```

#include <stdio.h>
#define MAX 100
#define INF 999999

```

```

int parent[MAX];

```

```

int find(int v){
    while(parent[v])
        v = parent[v];
    return v;
}

```

```

void kruskal(int cost[MAX][MAX], int n){
    int edges = 0, minCost = 0;

```

```

while(edges < n - 1){
    int min = INF, u = -1, v = -1;

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            if(cost[i][j] < min){
                min = cost[i][j];
                u = i;
                v = j;
            }

    int ru = find(u), rv = find(v);

    if(ru != rv){
        parent[rv] = ru;
        printf("%d - %d : %d\n", u, v, min);
        minCost += min;
        edges++;
    }

    cost[u][v] = cost[v][u] = INF;
}

printf("Minimum Cost = %d\n", minCost);
}

int main(){
    int n, cost[MAX][MAX];

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    printf("Enter the adjacency matrix:\n");

```

```

for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++){
        scanf("%d", &cost[i][j]);
        if(i == j || cost[i][j] == 0)
            cost[i][j] = INF;
    }

kruskal(cost, n);
return 0;
}

```

OUTPUT:

```

Enter the number of vertices: 4
Enter the adjacency matrix:
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0
2 - 3 : 4
0 - 3 : 5
0 - 2 : 6
Minimum Cost = 15

```

Lab program 9:

Fractional Knapsack

```
#include <stdio.h>
```

```
#define MAX 15
```

```
struct item{
```

```
    char name;
```

```
    int profit;
```

```
    int wt;
```

```
    float ratio;
```

```
}a[MAX];
```

```
void swap(struct item *a, int i, int j){
```

```
    int t[2];char ch;float f;
```

```
    t[0]=a[i].profit;
```

```
    t[1]=a[i].wt;
```

```
    ch = a[i].name;
```

```
    f=a[i].ratio;
```

```
    a[i].profit = a[j].profit;
```

```
    a[i].wt = a[j].wt;
```

```
    a[i].ratio = a[j].ratio;
```

```
    a[i].name = a[j].name;
```

```
    a[j].name = ch;
```

```
    a[j].profit = t[0];
```

```
    a[j].wt = t[1];
```

```
    a[j].ratio = f;
```

```
}
```

```
int sort(struct item* a,int st,int end){
```

```
    float p=a[st].ratio;
```

```
    int i=st,j=end;
```

```
    while (1) {
```

```

        while (i <= end && a[i].ratio >= p) i++;
        while (j >= st && a[j].ratio < p) j--;
        if (i >= j) break;
        swap(a, i, j);
    }
    swap(a, st, j);
    return j;
}

```

```

void quickSort(struct item* a, int st, int end ) {
    if(st >= end) return;

    int mid = sort(a, st, end);
    quickSort(a, st, mid-1);
    quickSort(a, mid+1, end);
}

```

```

void fillUp(struct item* a, int cap, int n) {
    quickSort(a, 0, n-1);
    float pro=0.0;
    for(int i=0; i<n && cap>0 ; i++){
        if(cap- a[i].wt >= 0){
            cap-=a[i].wt;
            pro+= a[i].profit;
        }
        else {
            pro += cap * a[i].ratio;
            cap=0;
        }
        printf("%c ", a[i].name);
    }
    printf("\nProfit = %.2f", pro);
}

```



```

int main(){
    printf("Enter size:");
    int n;
    scanf("%d",&n);

    printf("Enter wt, profit and name resp :\n");
    for(int i=0;i<n;i++){
        scanf("%d",&a[i].wt);
        scanf("%d",&a[i].profit);
        scanf(" %c",&a[i].name);
        a[i].ratio = (float)a[i].profit / a[i].wt;
    }
    int cap;
    printf("\nEnter capacity :");
    scanf("%d",&cap);
    fillUp(a, cap, n);
    return 0;
}

```

OUTPUT:

```

Enter size:3
Enter wt, profit and name resp :
10 60 a
20 100 b
30 120 c

Enter capacity :50
a ,b ,c ,
Profit = 240.00

```

LEETCODE

455. Assign Cookies Solved

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor $g[i]$, which is the minimum size of a cookie that the child will be content with; and each cookie j has a size $s[j]$. If $s[j] \geq g[i]$, we can assign the cookie j to the child i , and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:
Input: $g = [1,2,3]$, $s = [1,1]$
Output: 1
Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3. And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content. You need to output 1.

Example 2:
Input: $g = [1,2]$, $s = [1,2,3]$
Output: 2
Explanation: You have 2 children and 3 cookies. The greed factors of 2 children are 1, 2. You have 3 cookies and their sizes are big enough to gratify all of the children.

Runtime: 29 ms | Beats 13.19%
Memory: 12.52 MB | Beats 74.52%

```
1 int compare(const void *a, const void *b) {
2     return (*(int*)a - *(int*)b);
3 }
4 int findContentChildren(int* g, int gSize, int* s, int sSize) {
5     qsort(g, gSize, sizeof(int), compare);
6     qsort(s, sSize, sizeof(int), compare);
```

```
int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int findContentChildren(int* g, int gSize, int* s, int sSize) {
    qsort(g, gSize, sizeof(int), compare);
    qsort(s, sSize, sizeof(int), compare);

    int i = 0;
    int j = 0;
    while (i < gSize && j < sSize) {
        if (s[j] >= g[i]) i++;
        j++;
    }
    return i;
}
```

Lab program 10:

Dijkstra's Algorithm

```
#include <stdio.h>
```

```
#define V 10;
```

```
int minDist(int visited[], int dist[], int n){  
    int min = 10000, minIdx = -1;  
    for(int v=0;v<n;v++){  
        if(!visited[v] && dist[v]<min){  
            min= dist[v]; minIdx = v;  
        }  
    }  
    return minIdx;  
}
```

```
void Dijkstra(int n, int graph[n][n],int src){  
    int dist[n];  
    int visited[n];  
    for(int i=0;i<n;i++){  
        dist[i]= 10000;  
        visited[i]=0;  
    }
```

```
    dist[src] = 0;
```

```
    for(int i=1;i<n;i++){  
        int u= minDist(visited, dist , n);  
        visited[u] = 1;
```

```
        for(int v=0;v<n;v++){  
            if(!visited[v] && graph[u][v] && graph[u][v]+ dist[u] < dist[v])  
                dist[v] = dist[u] + graph[u][v];  
        }  
    }
```

```
    printf("Vertex\tDistance from Source :\n");
```

```

        for(int i=0;i<n;i++) printf("%d\t%d\n", i, dist[i]);

    }

void main(){
    int n;
    printf("Enter number of vertex:");
    scanf("%d", &n);

    int graph[n][n];
    printf("Enter weights:\n");
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            scanf("%d", &graph[i][j]);
    // int graph[n][n] = {
    //     {0, 10, 0, 30, 100},
    //     {10, 0, 50, 0, 0},
    //     {0, 50, 0, 20, 10},
    //     {30, 0, 20, 0, 60},
    //     {100, 0, 10, 60, 0}
    // };
    Dijkstra(n, graph, 0);
}

```

OUTPUT:

```

Enter number of vertex:5
Enter weights:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Vertex Distance from Source :
0 0
1 10
2 50
3 30
4 60

```

Lab Program 11:

N Queens Problem

```
#include <stdio.h>
#include <stdlib.h>
```

```
int x[20];
int sol = 0;
```

```
int place(int k, int i){
    for (int j = 1; j < k; j++)
        if (x[j] == i || abs(j - k) == abs(x[j] - i))
            return 0;
    return 1;
}
```

```
void NQueens(int k, int n){
    for (int i = 1; i <= n; i++) {
        if (place(k, i)){
            x[k] = i;
            if (k == n){
                sol++;
                printf("\nSolution %d:\n", sol);

                for (int row = 1; row <= n; row++) {
                    for (int col = 1; col <= n; col++){
                        if (x[row] == col)
                            printf("1 ");
                        else
                            printf("0 ");
                    }
                    printf("\n");
                }
            }
        }
    }
    else
```

```

        NQueens(k + 1, n);
    }
}

int main(){
    int n;
    printf("Enter number of queens: ");
    scanf("%d", &n);
    NQueens(1, n);
    printf("\nTotal solutions = %d\n", sol);
    return 0;
}

```

OUTPUT:

```
Enter number of queens: 4
```

```
Solution 1:
```

```
0 1 0 0
```

```
0 0 0 1
```

```
1 0 0 0
```

```
0 0 1 0
```

```
Solution 2:
```

```
0 0 1 0
```

```
1 0 0 0
```

```
0 0 0 1
```

```
0 1 0 0
```

```
Total solutions = 2
```